

Rata32: Robot Autónomo para la Resolución de Laberintos mediante Algoritmos de Búsqueda y Comunicación MQTT

*Rata32: Autonomous Maze-Solving Robot using
Search Algorithms and MQTT Communication*

Apellido, Nombre 1

Universidad Tecnológica Nacional, Facultad Regional San Francisco

email1@ejemplo.com

Apellido, Nombre 2

Universidad Tecnológica Nacional, Facultad Regional San Francisco

email2@ejemplo.com

Apellido, Nombre 3

Universidad Tecnológica Nacional, Facultad Regional San Francisco

email3@ejemplo.com

Ingeniería Electrónica

Universidad Tecnológica Nacional

Facultad Regional San Francisco

February 17, 2026

Resumen

Se presenta el desarrollo de Rata32, un robot autónomo para la resolución de laberintos basado en el microcontrolador ESP32. El sistema incorpora tres sensores ultrasónicos para la detección de obstáculos, encoders ópticos para la estimación de posición y un puente H L298N para el control de motores de corriente continua. Durante una primera pasada, el robot explora el laberinto de forma autónoma mientras transmite su trayectoria en tiempo real mediante el protocolo MQTT al broker HiveMQ. Una interfaz web recibe los datos, grafica el recorrido y, al detectar el punto final, aplica los algoritmos A*, Dijkstra o BFS para calcular el camino óptimo. En una segunda pasada, el robot recorre dicho camino de manera eficiente. El proyecto integra programación en C++ para el firmware, JavaScript para la lógica de resolución e interfaz gráfica, y HTML/CSS para la presentación visual. Los resultados obtenidos demuestran la resolución autónoma y satisfactoria del laberinto.

Palabras clave: robot autónomo, resolución de laberintos, ESP32, MQTT, algoritmos de búsqueda

Abstract

This paper presents the development of Rata32, an autonomous maze-solving robot based on the ESP32 microcontroller. The system incorporates three ultrasonic sensors for obstacle detection, optical encoders for position estimation, and an L298N H-bridge for DC motor control. During a first run, the robot autonomously explores the maze while transmitting its trajectory in real time via the MQTT protocol to the HiveMQ broker. A web interface receives the data, plots the path, and upon detecting the endpoint, applies A*, Dijkstra, or BFS algorithms to calculate the optimal route. In a second run, the robot efficiently follows this route. The project integrates C++ programming for the firmware, JavaScript for solving logic and graphical interface, and HTML/CSS for visual presentation. Results demonstrate successful and autonomous maze resolution.

Keywords: autonomous robot, maze solving, ESP32, MQTT, search algorithms

1. Introducción

1.1. Objetivos

- Desarrollar un robot autónomo capaz de explorar y resolver laberintos físicos sin intervención humana.
- Implementar y comparar algoritmos de búsqueda de caminos (A*, Dijkstra y BFS) sobre el mapa construido a partir de la trayectoria recorrida.
- Diseñar e implementar una interfaz web de monitoreo y control en tiempo real mediante el protocolo MQTT.
- Integrar sensores ultrasónicos y encoders ópticos para la navegación autónoma del robot.
- Visualizar en tiempo real la trayectoria recorrida y el camino óptimo calculado sobre un gráfico dinámico en la interfaz web.

1.2. Antecedentes

Los robots resuelve-laberintos tienen una larga historia en el campo de la robótica autónoma. La competencia Micromouse, organizada por el IEEE desde 1977, es uno de los eventos más reconocidos en esta área, donde robots de pequeñas dimensiones deben encontrar el camino más corto hacia el centro de un laberinto de forma completamente autónoma. Este evento ha impulsado el desarrollo de algoritmos eficientes de navegación y exploración, siendo Flood Fill y A* los más utilizados en competencias de alto nivel.

En el marco de la investigación previa para este proyecto, se analizaron videos de competencias Micromouse, repositorios públicos en GitHub con implementaciones similares, y las hojas de datos técnicas de los componentes utilizados: el microcontrolador ESP32, el sensor ultrasónico HC-SR04 y el puente H L298N. A diferencia de los proyectos relevados, el diseño mecánico, electrónico y de software del presente trabajo es de elaboración propia, adaptado a los recursos disponibles y a los contenidos de la asignatura.

2. Desarrollo

2.1. Descripción general del sistema

Rata32 es un robot móvil autónomo de tracción diferencial diseñado para explorar y resolver laberintos físicos. El sistema opera en dos fases: durante la primera, el robot recorre el laberinto de forma autónoma utilizando un algoritmo seguidor de pared derecha, transmitiendo su posición en tiempo real a una interfaz web mediante el protocolo MQTT. Durante la segunda fase, la interfaz web calcula el camino óptimo sobre el mapa construido y el robot lo recorre de forma eficiente.

El sistema integra cinco subsistemas principales: el módulo de locomoción, el módulo de percepción, el módulo de odometría y control, el módulo de comunicación y la interfaz web de monitoreo y control.

2.2. Hardware

2.2.1 Microcontrolador ESP32

El microcontrolador utilizado es el ESP32, un sistema en chip (SoC) de doble núcleo que opera a 240 MHz, con conectividad WiFi 802.11 b/g/n integrada. Su arquitectura de doble núcleo resulta fundamental para el proyecto, ya que permite distribuir las tareas críticas de control en el Core 0 y las tareas de sensado y comunicación en el Core 1, ejecutándose de forma concurrente mediante el sistema operativo en tiempo real FreeRTOS.

2.2.2 Sensores ultrasónicos HC-SR04

Se utilizan tres sensores ultrasónicos HC-SR04 ubicados en las posiciones frontal, lateral derecha y lateral izquierda del robot. Cada sensor emite un pulso ultrasónico de 40 kHz y mide el tiempo de retorno del eco para calcular la distancia a los obstáculos, con un rango efectivo de 2 cm a 400 cm. Las lecturas se realizan de forma secuencial con un intervalo mínimo de 30 ms entre sensores para evitar interferencias entre ellos.

2.2.3 Sensor infrarrojo

Para la detección del punto final del laberinto se utiliza un sensor infrarrojo analógico conectado al pin 35 de la ESP32. El suelo del laberinto presenta una cuadrícula final de color negro sobre fondo blanco. Cuando el valor analógico del sensor supera el umbral de 2000, el sistema determina que el robot alcanzó el final del laberinto, detiene los motores y notifica a la interfaz web mediante MQTT.

2.2.4 Encoders ópticos

Cada rueda motriz cuenta con un disco ranurado impreso en 3D solidario al eje del motor. Un sensor óptico de ranura detecta el paso de cada ranura generando pulsos digitales. Con 30 pulsos por revolución y un diámetro de rueda de 5 cm, cada pulso equivale a 0,524 cm de desplazamiento. Las interrupciones de los encoders se procesan en la memoria IRAM de la ESP32 para garantizar la máxima velocidad de respuesta, protegidas por secciones críticas mediante un mutex de FreeRTOS.

2.2.5 Puente H L298N y motores DC

El control de los motores se realiza mediante un módulo L298N, un puente H de doble canal capaz de manejar hasta 2 A por canal. Se utilizan dos motores DC con reductora (motores amarillos estándar de 3V–6V), uno por rueda motriz, configurados en tracción diferencial, y una rueda loca en la parte trasera para soporte. La generación de la señal PWM se realiza mediante el módulo LEDC de la ESP32 a una frecuencia de 15 kHz y resolución de 8 bits, frecuencia elegida para eliminar el ruido audible producido por frecuencias menores. El chasis y las ruedas fueron diseñados en FreeCAD e impresos en 3D, incluyendo los discos ranurados de los encoders.

2.3. Firmware (C++)

2.3.1 Estructura del código

El firmware fue desarrollado en C++ sobre el entorno Arduino IDE utilizando el framework ESP32 con soporte para FreeRTOS. La arquitectura del sistema se basa en cinco tareas concurrentes distribuidas entre los dos núcleos del ESP32, comunicadas mediante colas y semáforos de FreeRTOS.

El Core 0 ejecuta la tarea de mayor criticidad temporal: el controlador PID y la odometría, con una frecuencia de 200 Hz. El Core 1 gestiona las tareas de menor prioridad temporal: lectura de sensores ultrasónicos a 16 Hz, lectura del sensor infrarrojo a 20 Hz, navegación autónoma a 7 Hz y comunicación MQTT a 10 Hz. Esta distribución garantiza que el lazo de control no se vea interrumpido por operaciones de red o sensado.

La comunicación entre tareas se implementa mediante cuatro colas FreeRTOS: una cola de sensores (ultrasonidos hacia navegación), una cola de comandos (MQTT hacia motores), una cola de posición (odometría hacia MQTT) y una cola de telemetría (cualquier tarea hacia MQTT). Los accesos concurrentes a los contadores de pulsos de los encoders se protegen mediante secciones críticas con un mutex tipo `portMUX_TYPE`.

2.3.2 Control de motores y PID

Cada motor DC es controlado mediante un canal del módulo LEDC de la ESP32, configurado a 15 kHz y resolución de 8 bits. La frecuencia de 15 kHz fue seleccionada deliberadamente para superar el umbral de audición humano, eliminando el zumbido característico de frecuencias menores. La dirección de cada motor se controla mediante dos pines digitales conectados al L298N.

El control de velocidad implementa un controlador PID independiente para cada rueda, con los parámetros $K_p = 2,0$, $K_i = 0,5$ y $K_d = 0,1$. El término integral incorpora un mecanismo anti-windup que limita su acumulación al rango $[-50, 50]$ para evitar la saturación del actuador. El PID opera únicamente en modo manual; durante la navegación autónoma los integrales

se resetean para evitar interferencias con la lógica de navegación.

2.3.3 Odometría

La estimación de posición se realiza mediante odometría diferencial. En cada ciclo de 5 ms se calculan los desplazamientos individuales de cada rueda a partir de la variación de pulsos de los encoders, aplicando la ecuación:

$$d_i = \Delta pulsos_i \times \frac{\pi \cdot D}{N} \quad (1)$$

donde $D = 5$ cm es el diámetro de la rueda y $N = 30$ es el número de pulsos por revolución. La posición en el plano cartesiano se actualiza mediante transformación vectorial:

$$\Delta\theta = \frac{d_{der} - d_{izq}}{L}, \quad \Delta x = d_c \cdot \cos(\theta), \quad \Delta y = d_c \cdot \sin(\theta) \quad (2)$$

donde $L = 15$ cm es la distancia entre ruedas y d_c es la distancia recorrida por el centro del robot. Las coordenadas calculadas se publican por MQTT a 50 Hz para su visualización en tiempo real.

2.3.4 Lectura de sensores

Los tres sensores HC-SR04 se leen de forma secuencial con un intervalo de 30 ms entre cada lectura para evitar interferencias acústicas entre sensores. La distancia se calcula como:

$$d = \frac{t_{eco} \times 0,034}{2} \quad (3)$$

donde t_{eco} es la duración del pulso de eco en microsegundos. Si no se recibe eco en 30 ms, la distancia se reporta como 999 cm indicando camino libre. El sensor infrarrojo se lee mediante el conversor analógico-digital de la ESP32, comparando el valor contra un umbral de 2000 para distinguir entre suelo blanco y negro.

2.3.5 Comunicación MQTT

La comunicación se establece mediante el protocolo MQTT sobre TLS (puerto 8883) con el broker HiveMQ Cloud. Se definen tres tópicos: `robot01/cmd` para recibir comandos desde la web, `robot01/data` para enviar coordenadas y eventos, y `robot01/status` para publicar el estado completo del sistema. La reconexión automática se reintenta cada 5 segundos ante pérdida de conectividad.

2.4. Interfaz web

2.4.1 Arquitectura de la página

La interfaz web fue desarrollada con HTML5, CSS3 y JavaScript puro, sin frameworks adicionales. El código se organiza en módulos independientes: `mqtt.js` gestiona la conexión con el broker, `maze.js` implementa el visualizador de trayectoria sobre canvas HTML5, `solve.js` contiene los algoritmos de resolución, `ui.js` maneja los controles de usuario y `config.js` centraliza los parámetros de configuración.

2.4.2 Comunicación con el broker HiveMQ

La página se conecta al broker HiveMQ Cloud mediante el protocolo MQTT sobre WebSocket seguro (WSS), utilizando la biblioteca Paho MQTT para JavaScript. Al recibir datos en el tópico `robot01/data`, el módulo MQTT parsea las coordenadas y las envía al visualizador. Los comandos de control (adelante, atrás, izquierda, derecha, stop, modo autónomo) se publican en el tópico `robot01/cmd`.

2.4.3 Visualización de trayectoria

El módulo `maze.js` implementa un visualizador dinámico sobre un elemento canvas de HTML5. Cada coordenada recibida por MQTT se agrega como un punto al recorrido, redibujando el canvas en tiempo real. El punto de inicio se marca en verde al recibir la primera coordenada, y el punto final en rojo cuando el robot detecta la línea negra y envía el evento

LINEA_NEGRA_DETECTADA. El visualizador incluye una grilla de referencia cada 5 cm, información de distancia total recorrida y rango de coordenadas, con escala y grosor de línea configurables.

2.4.4 Algoritmos de resolución

Antes de iniciar la exploración, el usuario selecciona mediante la interfaz web el algoritmo de resolución a utilizar, pudiendo optar entre A*, Dijkstra o BFS. Una vez que el robot completa la exploración y detecta el punto final, el módulo `solve.js` construye una grilla discreta a partir de los puntos registrados, marcando como camino libre las celdas transitadas mediante el algoritmo de Bresenham para el trazado de líneas. Sobre esta grilla se ejecuta el algoritmo previamente seleccionado para calcular el camino óptimo.

El algoritmo A* utiliza la distancia Manhattan como función heurística, explorando primero los nodos con menor valor de $f(n) = g(n) + h(n)$, donde $g(n)$ es el costo acumulado desde el inicio y $h(n)$ la estimación al destino. Garantiza el camino óptimo con menor cantidad de nodos explorados que Dijkstra.

El algoritmo de Dijkstra explora todos los nodos accesibles en orden creciente de costo acumulado, sin función heurística. Garantiza el camino óptimo pero explora más nodos que A*.

La búsqueda en anchura (BFS) explora el grafo nivel por nivel desde el nodo inicial. En grafos con costos uniformes como este, también garantiza el camino óptimo y sirve como referencia de comparación. El camino calculado se superpone sobre el recorrido del robot en el canvas con una línea de color diferente, permitiendo la comparación visual entre la trayectoria explorada y el camino óptimo.

3. Resultados

4. Conclusiones

Agradecimientos

Referencias